

TMDS: Temperature-aware Makespan Minimizing DAG Scheduler for Heterogeneous Distributed Systems

DEBABRATA SENAPATI, Indian Institute of Technology (IIT) Guwahati, India and SRM University - AP, Andhra Pradesh, India
KOUSIK RAJESH and CHANDAN KARFA, Indian Institute of Technology (IIT) Guwahati, India
ARNAB SARKAR, Indian Institute of Technology (IIT) Kharagpur, India

To meet application-specific performance demands, recent embedded platforms often involve the use of intricate micro-architectural designs and very small feature sizes leading to complex chips with multi-million gates. Such ultra-high gate densities often make these chips susceptible to inappropriate surges in core temperatures. Temperature surges above a specific threshold may throttle processor performance, enhance cooling costs, and reduce processor life expectancy. This work proposes a generic temperature management strategy that can be easily employed to adapt existing state-of-the-art task graph schedulers so that schedules generated by them never violate stipulated thermal bounds. The overall temperature-aware task graph scheduling problem has first been formally modeled as a constraint optimization formulation whose solution is shown to be prohibitively expensive in terms of computational overheads. Based on insights obtained through the formal model, a new fast and efficient heuristic algorithm called *TMDS* has been designed. Experimental evaluation over diverse test case scenarios shows that *TMDS* is able to deliver lower schedule lengths compared to the temperature-aware versions of four prominent *makespan* minimizing algorithms, namely, *HEFT*, *PEFT*, *PPTS*, and *PSLS*. Additionally, a case study with an adaptive cruise controller in automotive systems has been included to exhibit the applicability of *TMDS* in real-world settings.

CCS Concepts: • **Computer systems organization → Embedded and cyber-physical systems;**

Additional Key Words and Phrases: DAG scheduling, temperature-aware, makespan, list scheduling, heterogeneous platforms, distributed systems

ACM Reference format:
Debabrata Senapati, Kousik Rajesh, Chandan Karfa, and Arnab Sarkar. 2023. TMDS: Temperature-aware Makespan Minimizing DAG Scheduler for Heterogeneous Distributed Systems. *ACM Trans. Des. Autom. Electron. Syst.* 28, 6, Article 99 (October 2023), 22 pages.
https://doi.org/10.1145/3616869

Authors' addresses: D. Senapati, Department of Computer Science and Engineering, Indian Institute of Technology (IIT) Guwahati, Guwahati, Assam, 781039, India and SRM University AP, Andhra Pradesh - 522502, India; e-mail: debabratasenapati@gmail.com; K. Rajesh and C. Karfa, Department of Computer Science and Engineering, Indian Institute of Technology (IIT) Guwahati, Guwahati, Assam, 781039, India; e-mails: kousikr26@gmail.com, ckarfa@iitg.ac.in; A. Sarkar, Advanced Technology Development Centre, Indian Institute of Technology (IIT) Kharagpur, Kharagpur, West Bengal, 721302, India; e-mail: arnab@atdc.iitkgp.ac.i;
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
1084-4309/2023/10-ART99 $15.00
https://doi.org/10.1145/3616869

1 INTRODUCTION

Applications in modern embedded processors are often modeled as **Directed Acyclic Graphs (DAGs)** (alternatively referred to as task graphs in the remainder of this article) due to their distributed nature and interactions among component functionalities. In such DAGs, nodes depict tasks, while directed edges indicate inter-dependencies among task pairs. To meet functionality-specific high performance and reliability requirements, these DAGs are typically implemented on heterogeneous platforms [22]. On these heterogeneous execution platforms, a task may require different computation times on different processors, and inter-task messages having the same quantity of data may incur specific transmission times on the various communication channels due to heterogeneous channel bandwidths [21].

The introduction of sub-micron VLSI advancements has led to highly dense multi-million gates per chip, where power dissipation rates and thermal management have become critical design issues [29]. Unconfined surges in temperature not only increase cooling costs but may also reduce system performance and life expectancy. Authors in Reference [34] have shown that a chip's life span could be decreased by up to 50% with a 10 °C–15 °C temperature rise beyond normal operating temperature. Therefore, modern multicore embedded systems generally come with a specified temperature threshold that must be adhered to for safe and efficient system operation. ***Dynamic Thermal Management*** **(DTM)** techniques that enforce performance throttling to alleviate temperature hotspots, are set off whenever the operating temperature exceeds the predefined threshold temperature in the system [15]. The DTM triggers lead to operations like powering-down processors [33], clock or fetch gating [28], **Dynamic Voltage and Frequency Scaling (DVFS)** [32, 36], and so on. Almost all modern processors manage these methods at the hardware level having multiple power domains and operating at different frequency/voltage settings. However, the DTM techniques typically focus solely on the lowering of chip temperature, thus impacting application performance leading to the delivery of degraded **Quality of Service (QoS)**. Therefore, there is a need for platform resource managers that conduct task-to-processor assignments with suitably tuned configurations such that system-level temperature requirements are met. *Given a DAG-structured application with known thermal characteristics on a heterogeneous processing platform, successfully satisfying execution requirements of application tasks while ensuring that all processors operate within a specified temperature threshold is ultimately a scheduling problem.*

In general, the problem of DAG scheduling has been shown to be NP-complete [31]. The exponential state-space makes it infeasible to design an exact algorithm even for moderately large-size problems. Therefore, research in this domain usually focuses on developing low-overhead heuristics that deliver quick and acceptable schedules. Traditionally, list scheduling heuristics are known to generate efficient schedules for DAGs [1, 21–23, 30]. Most list scheduling approaches attempt to build a static schedule to minimize the overall *makespan* (or schedule length). This work attempts to design a *list-based heuristic scheduler to minimize the schedule makespan of task graphs by deciding task-processor mappings on a heterogeneous distributed platform without violating temperature-related specifications.*

The contributions of this article are summarized as follows:

(1) We proposed a generic temperature management strategy that can be easily employed to adapt existing state-of-the-art *makespan* minimizing DAG scheduling algorithms so that schedules generated by them never violate the processors' threshold temperatures.
(2) The temperature-aware DAG scheduling problem has been mathematically formulated as a constrained optimization problem, whose solution is shown to be prohibitively expensive in terms of computational overheads.

(3) Based on insights observed through the constraint optimization formulation, we endeavor to design a low-overhead list-based heuristic algorithm called, ***Temperature-aware Makespan Minimizing DAG Scheduler for Heterogeneous Distributed Systems (TMDS)***, for the problem at hand.
(4) Experimental evaluation using real-world benchmarks shows that the *TMDS* delivers lower schedule lengths compared to temperature-aware version of five *makespan* minimizing algorithms, *HEFT* [30], *PEFT* [1], *PPTS* [7], *PSLS* [37], and *ODS* [9], over various test scenarios.
(5) A case study on an adaptive cruise controller in automotive domains is used to exhibit the relevance of *TMDS* in realistic environments.

The rest of the article is organized as follows. Section 2 discusses related works. Section 3 presents system models and Section 4 describes the problem formulation. Section 5 discusses the proposed scheduling policy. Section 6 presents the detailed experimental results. Section 7 exhibits a real-world case study on an automotive control system. Finally, the conclusion of this article is presented in Section 8.

2 RELATED WORKS

Thermal hotspots have a significant impact on the performance and reliability of a system. Therefore, the processor's temperature management has become a topic of great concern for researchers and practitioners over the past few years. A survey of a few important temperature-aware scheduling techniques for multi-core processing systems may be found in Reference [27]. Huang et al. in Reference [8] proposed two throughput management policies for uni-processor systems with given temperature limits. The first policy shuffles processors between sleep and active states to meet the goal. The second policy recommends a scheduling technique considering the task's thermal characteristics and the processor's sleep/active levels. Bampis et al. [2] presented the *makespan* minimization problem under a given temperature bound. They studied the temperature-aware scheduling problem for systems consisting of unit-length independent tasks and homogeneous processors with known thermal characteristics. Authors in Reference [3] addressed a thermal-aware global multiprocessor scheduling algorithm to increase the number of finished tasks. They examine the competitive ratios of a large class of scheduling techniques as a function of the processors' cooling factors. In Reference [38], Zhou et al. presented a heuristic strategy for scheduling independent tasks with reliability and temperature constraints on heterogeneous platforms to minimize schedule length. A few recent scheduling techniques on temperature management that target independent tasks on homogeneous and heterogeneous systems may be found in References [17, 18] and [15, 24], respectively.

Research works on the temperature-aware scheduling of precedence-constrained task graphs are relatively fewer. Sheikh et al. [25] proposed a temperature-aware scheduling algorithm to minimize the *makespan* of DAG-structured applications. In Reference [26], the same authors also proposed a DAG scheduling technique for simultaneously optimizing *makespan*, energy and temperature. However, both the works are focused towards homogeneous multi-cores.

Over the years, the problem of scheduling DAGs on heterogeneous systems has often been dealt with *list scheduling heuristics* [1, 4, 6, 7, 10, 11, 30, 37] with *makespan* minimization being the primary scheduling objectives. List scheduling algorithms typically incorporate two phases, (i) *Task Prioritization*: picking the highest-priority ready task and (ii) *Processor Selection*: deciding an appropriate processor for each task to minimize overall schedule length. Topcuoglu et al. [30] presented a list scheduling policy called *HEFT*, which determines the priority of each task via *upward rank*. The *upward rank* of task τ_j is an estimate of the computation overhead of the path from τ_j to the sink task τ_{exit} of a task graph (including the execution overhead of τ_j). The processor

selection phase of *HEFT* picks a task having the highest priority and allocates it to a processor that minimizes its finish time. Canon et al. [5] compared 20 list scheduling algorithms and concluded that *HEFT* outperforms the others for schedule length. Later, a few more list scheduling algorithms: *HPS* [11], *PETS* [10], *LDCP* [6] and *Lookahead* [4] are proposed, and these policies outperform *HEFT* with respect to schedule length, albeit at the cost of higher computational overhead. In Reference [1], Arabnejad et al. proposed *PEFT*, which outperforms *HEFT*, *HPS*, *PETS*, *LDCP*, and *Lookahead* over a majority of test cases, while incurring quadratic computational overhead in terms of the number of tasks. Authors in References [7] and [37] proposed *PPTS* and *PSLS* as enhancements over the *PEFT* algorithm. Authors in Reference [9] proposed an enhancement to *HEFT*, called ***Out Degree Scheduling* (*ODS*)**.

In spite of the above efforts, there still exists a severe dearth of temperature-aware *makespan* minimizing DAG scheduler over distributed heterogeneous processing platforms. In this work, we have analyzed the performance of *TMDS* against temperature-aware extensions of four existing *makespan* minimizing algorithms, namely, *HEFT* [30], *PEFT* [1], *PPTS* [7], and *PSLS* [37].

3 SYSTEM MODELS

In this section, we present the application and platform model followed by temperature model.

3.1 Application and Platform Model

DAG $G(V, E)$ is a common way to represent an application where the set of vertices $V = \{\tau_1, \tau_2 \ldots, \tau_{|V|}\}$ denotes tasks and the set of directed edges E represents the precedence-constraints between task pairs. Each edge $e_{i,j}$ is marked with a positive weight $data_{i,j}$ indicating size of the message to be transmitted. We assume that a DAG has a single source/sink node. In the situation where a DAG has more than one source/sink node, a dummy source/sink node with dummy edges connecting to actual source/sink nodes is added. The set of all direct successors/predecessors of a task say, τ_j is depicted as $succ(\tau_j)/pred(\tau_j)$.

The platform $P = \{p_1, p_2, \ldots, p_{|P|}\}$ consists of $|P|$ fully interconnected heterogeneous processors. On these heterogeneous processors, each task possibly has a different **Worst Case Execution Time (*WCET*)** on different processors. For example, the WCET of each task of the DAG in Figure 1(a) on three heterogeneous processors are shown in Figure 1(b). An element $\omega_{j,n}$ stores the WCET of task τ_j on p_n. A matrix B of size $|P| \times |P|$ is defined to store heterogeneous bandwidths of links between all pairs of processors. An element $b_{m,n} \in B$ represents the data transfer rate between processors p_m and p_n, and the communication links are bidirectional, i.e., $b_{m,n}$ equals to $b_{n,m}$. The communication time between tasks τ_i (executing on processor p_m) and τ_j (executing on p_n; $\tau_j \in succ(\tau_i)$) can be determined as

$$c_{i,j}^{m,n} = \begin{cases} 0, & if\, m = n, \\ data_{i,j}/b_{m,n}, & otherwise. \end{cases} \tag{1}$$

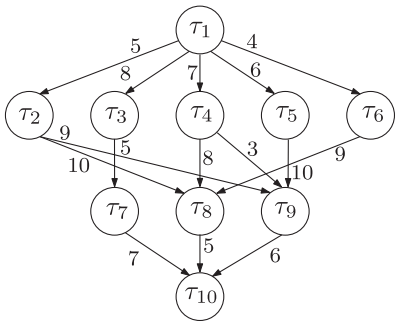
The above equation reveals that the communication time is considered to be negligible when both the tasks τ_i and τ_j are allocated to the same processor.

3.2 Temperature Model

The rate of change in temperature (in °C) of a processor is modeled similar to References [8, 17]:

$$\frac{d\,\Gamma^t}{dt} = \frac{pow^t}{C} - \frac{\Gamma^t - \Gamma^a}{C\,R}, \tag{2}$$

where (i) Γ^t is the processor's temperature at time instant t, (ii) Γ^a is the ambient temperature, (iii) pow^t represents the power dissipation (in W) at time t, and (iv) C and R indicate the thermal



(a) A DAG $G(V, E)$

Tasks	$\omega_{j,n}$			$\Gamma^{ss}_{j,n}$		
	p_1	p_2	p_3	p_1	p_2	p_3
τ_1	16	15	14	107	94	84
τ_2	16	17	18	105	98	83
τ_3	18	16	13	100	93	82
τ_4	19	10	17	101	95	82
τ_5	19	18	20	104	94	81
τ_6	15	16	13	103	94	85
τ_7	20	18	12	104	93	86
τ_8	16	12	14	102	92	83
τ_9	20	19	17	99	85	84
τ_{10}	15	12	7	107	98	83

(b) WCET and Steady State temperature

Fig. 1. (a) DAG with 10 tasks; (b) WCETs and steady-state temperature of 10 tasks on 3 processors.

capacitance (in $J/°C$) and resistance (in $°C/W$) of a processor, respectively. Scaling Γ^t such that Γ^a is *zero*, Equation (2) gets reduced to Equation (3):

$$\frac{d\,\Gamma^t}{dt} = a\,pow^t - b\,\Gamma^t, \tag{3}$$

where $a = 1/C$ and $b = 1/CR$. Here, pow^t comprises of two factors: **dynamic power dissipation** (pow^d), which is independent of temperature and the **leakage power dissipation** (pow^l), which is dependent on the processor's temperature [8, 17, 32]:

$$pow^t = pow^d + pow^l = C_0 f^3 + (C_1 f + C_2 f\,\Gamma^t), \tag{4}$$

where C_0, C_1, and C_2 are constants that are dependent on processor's frequency f (proportional to the voltage [16]). Here, we assume that each processor can run only at a single frequency say, f. Plugging Equation (4) into Equation (3), the rate of change in processor temperature can be formulated as

$$\frac{d\,\Gamma^t}{dt} = a[C_0 f^3 + C_1 f + C_2 f\,\Gamma^t] - b\,\Gamma^t = \mathcal{A} - \mathcal{B}\Gamma^t, \tag{5}$$

where $\mathcal{A} = a(C_0 f^3 + C_1 f)$ and $\mathcal{B} = (b - aC_2 f)$. For a time interval $[0, q]$ in which task τ_j is executing on processor p_n, if the processor temperature at the start of the interval is Γ^0 and Γ^q at the end of the interval, then Equation (5) can be rewritten as

$$\Gamma^q = \mathcal{A}/\mathcal{B} + (\Gamma^0 - \mathcal{A}/\mathcal{B})e^{-\mathcal{B}(q-0)}. \tag{6}$$

We now define a function $T_{heat}()$ to determine the temperature Γ^q of a processor p_n at time q when a task τ_j continually executes on it starting from time 0 with temperature Γ^0:

$$T_{heat}\left(\Gamma^{ss}_{j,n}, \Gamma^0, q-0\right) = \Gamma^{ss}_{j,n} + \left(\Gamma^0 - \Gamma^{ss}_{j,n}\right) e^{-\mathcal{B}_n(q-0)}, \tag{7}$$

where $\mathcal{B}_n = (1 - RC_2 f)/CR$ represents the thermal characteristics of processor p_n and $\Gamma^{ss}_{j,n}$ is the steady-state temperature of a task τ_j on p_n. $\Gamma^{ss}_{j,n}$ denotes the temperature that a processor may reach when τ_j executes continuously on p_n for a sufficiently long time. Thus, $\Gamma^{ss}_{j,n} = \Gamma^a + \mathcal{A}_n/\mathcal{B}_n$. A very similar function can be defined to obtain the temperature of p_n when it remains idle for a time interval $[0, q]$, starting from its initial temperature Γ^0 at time 0:

$$T_{cool}(\Gamma^a, \Gamma^0, q-0) = \Gamma^a + (\Gamma^0 - \Gamma^a)e^{-\mathcal{B}_n(q-0)}. \tag{8}$$

Some of the important notations and their meanings have been listed in Table 1.

The objective of this work is to design a DAG scheduling algorithm that attempts to minimize makespan, while ensuring that processor temperatures never overshoot their respective threshold cutoff

Table 1. List of Important Notations Used and Their Meanings

Symbols	Descriptions	Acronyms	Full Forms
τ_j	jth task	DTM	Dynamic Thermal Management
p_n	nth processor	WCET	Worst Case Execution Time
$\omega_{j,n}$	WCET of task τ_j on processor p_n	EST	Effective Start Time
t	Current time	EFT	Effective Finish Time
Γ^a	Ambient room temperature	AST	Actual Start Time
Γ^0	Initial temperature	AFT	Actual Finish Time
Γ^q	Final temperature	OFT	Optimistic Finish Time
$\Gamma^{ss}_{j,n}$	Steady-state temperature of task τ_j on processor p_n	TETT	Task Execution Time and final Temperature
$\Gamma^{st}_{j,n}$	Starting temperature of task τ_j on processor p_n		
$\mathcal{B}_n$	Thermal characteristics of processor p_n	MDE	Maximum Duration of continuous Execution
Γ^{lim}_n	Upper bound on temperature associated with p_n		
$\Gamma^c_{j,n}$	Cutoff temperature of τ_j on p_n	SCT	Schedule Completion Time
Γ^t_n	Temperature of a processor p_n at any time step t	OV	Overhead
$\Gamma^t_{j,n}$	p_n's temperature at time t when τ_j is mapped on it	SLR	Schedule Length Ratio
T	Upper bound on possible schedule length	CCR	Communication-to-Computation Ratio
$\xi_{j,n}$	Duration in which task τ_j remains allocated on p_n		
S_j	Start time of task τ_j	ACC	Adaptive Cruise Controller

values. In systems where the steady-state temperatures of one or more tasks (on some or all processors) are higher than their threshold cutoffs, the above objective is achieved by opportunistically inserting idle intervals (to allow processor cooling) within the schedule.

4 PROBLEM FORMULATION

In this section, we formally model the above discussed problem as a *Constraint Optimization Problem*. The formulation uses two sets of decision variables: (i) $X = \{x^t_{j,n} : j = 1, 2, \ldots, |V|; n = 1, 2, \ldots, |P|; t = 0, 1, \ldots, T\}$. Here, $T = \sum_{j=0}^{|V|} \max_{p_n \in P} \omega_{j,n}$ denotes the upper bound on possible schedule length. The term $\omega_{j,n}$ denotes the WCET of a task τ_j on a processor p_n and is defined as the maximum time taken when the task runs standalone in a control setup when the processor temperature can never exceed its stipulated threshold upper bound. $x^t_{j,n} = 1$, indicates that task τ_j is mapped to processor p_n and starts execution from the tth time step; otherwise, $x^t_{j,n} = 0$. (ii) $Z = \{z^\alpha_{j,n} : j = 1, 2, \ldots, |V|; n = 1, 2, \ldots, |P|; \alpha = 0, 1, \ldots, T\}$. $z^\alpha_{j,n} = 1$ indicates that task τ_j should be executing on processor p_n at time step α subsequent to the start of τ_j; $z^\alpha_{j,n} = 0$ means one of two possible cases: (i) Processor p_n is idling with task τ_j is allocated on p_n. This case happens when $\alpha < \xi_{j,n}$. Here, $\xi_{j,n} = \max_{\alpha=1}^{T} \alpha \; z^\alpha_{j,n}$. The value of $\xi_{j,n}$ represents the duration during which task τ_j remains allocated on processor p_n. (ii) Task τ_j is not allocated on p_n. This happens when $\alpha > \xi_{j,n}$. Next, we present the objective function and the necessary constraints on the binary decision variables to model the scheduling problem.

Objective Function: Our objective is to minimize the overall schedule length while satisfying all scheduling constraints. The objective function can be defined as

$$Minimize \sum_{n=1}^{|P|} \sum_{t=0}^{T} x^t_{exit,n} \times (t + \xi_{exit,n}), \tag{9}$$

subject to satisfaction of the constraints in Equations (10)–(14).

Unique Start Time Constraints: The start time of each task on a specific processor should be unique; that is,

$$\forall j \in [1, |V|],\ \sum_{n=1}^{|P|} \sum_{t=0}^{T} x_{j,n}^{t} = 1. \tag{10}$$

The above equation ensures that each task τ_j must starts its execution at a unique time step t exactly in one processor p_n. The start time of τ_j may be obtained as $S_j = \sum_{t=0}^{T} t \times x_{j,n}^{t}$.

Dependency Constraints: This enforces the satisfaction of precedence relationships among tasks within a DAG. That is, the execution start time of a task τ_j must be greater or equal to the arrival time of output messages among all the predecessors of τ_j. For any given predecessor τ_i, its arrival time is determined as the summation of τ_i's execution completion time (that includes cooling time) and the data transmission time from τ_i to τ_j:

$$\forall (\tau_i, \tau_j) \in E,\ \sum_{n=1}^{|P|} \sum_{t=0}^{T} t \times x_{j,n}^{t} \geqslant \sum_{n=1}^{|P|} \sum_{t=0}^{T} \sum_{m=1}^{|P|} \sum_{t'=0}^{T} x_{i,m}^{t'} \times x_{j,n}^{t} \left(t' + \xi_{i,m} + c_{i,j}^{m,n}\right). \tag{11}$$

Resource Constraints: Resource constraints enforce that a processor p_n can be allocated to at most one task at any given time. This constraint may be represented as

$$\forall n \in [1, |P|], \forall t \in [0, T],\ \lambda_n^t \leqslant 1. \tag{12}$$

It may be noted that a task τ_j can only be mapped on p_n for execution at time instant t, if it has started at most "$t - \xi_{j,n} + 1$" time steps earlier. In the above equation, $\lambda_n^t = \sum_{j=1}^{|V|} \rho_{j,n}^t$ and $\rho_{j,n}^t = \sum_{l=t-\xi_{j,n}+1}^{t} x_{j,n}^l$. The term $\rho_{j,n}^t$ assumes a value of "1" when τ_j starts on p_n between time intervals $[t - \xi_{j,n} + 1, t]$.

Execution Demand Constraints: Over the duration $\xi_{j,n}$, the total number of time slots in which τ_j should be executing on processor p_n must be exactly equal to $\omega_{j,n}$. That is,

$$\forall j \in [1, |V|], \forall n \in [1, |P|], \sum_{\alpha=1}^{\xi_{j,n}} z_{j,n}^{\alpha} = \omega_{j,n}. \tag{13}$$

Processor Temperature Constraints: The operating temperature (Γ_n^t) of a processor p_n at any time step t should never exceed a stipulated upper bound on temperature (Γ_n^{lim}) associated with p_n. That is,

$$\forall t \in [0, T], \forall n \in [1, |P|],\ \Gamma_n^t \leqslant \Gamma_n^{lim}. \tag{14}$$

Here, Γ_n^t is determined as follows:

$$\Gamma_n^t = \begin{cases} \Gamma^a\ , & if\ t = 0, \\ \Gamma^a + (\Gamma_n^{t-1} - \Gamma^a)e^{-\mathcal{B}_n}, & if\ t > 0\ \&\ \lambda_n^t = 0, \\ \sum\limits_{j=1}^{|V|} \Gamma_{j,n}^t\ , & otherwise. \end{cases} \tag{15}$$

In the above equation, Γ^a represents the ambient room temperature. If no task is executing (indicated by $\lambda_n^t = 0$) on processor p_n at any time t (> 0), then Γ_n^t is represented by the second line of Equation (15). Given Γ_n^{t-1} ($< \Gamma_{j,n}^{ss}$), the temperature of processor p_n at time $t - 1$, the expression in the **right-hand side (RHS)** produces the temperature Γ_n^t of p_n when it idles (thus, cools off) at the tth time slot (refer to Equation (8)). In the scenario when some task τ_j is allocated on p_n, two possible cases may arise: (i) p_n is executing τ_j, or (ii) p_n remains idle (for cool-off). In the third

line of Equation (15), $\Gamma^t_{j,n}$ indicates the temperature of p_n at time t when task τ_j is mapped on it. Equation (16) below determines $\Gamma^t_{j,n}$:

$$\Gamma^t_{j,n} = \begin{cases} 0, & if\ \rho^t_{j,n} = 0, \\ \{\Gamma^a + (\Gamma^{t-1}_n - \Gamma^a)e^{-\mathcal{B}_n}\} \times (1 - z^\alpha_{j,n}) \ + \{\Gamma^{ss}_{j,n} + (\Gamma^{t-1}_n - \Gamma^{ss}_{j,n})e^{-\mathcal{B}_n}\} \times z^\alpha_{j,n}, & otherwise. \end{cases} \tag{16}$$

The first line in Equation (16) says that $\Gamma^t_{j,n}$ assumes the value *zero*, when τ_j is not mapped on p_n (in this case, $\rho^t_{j,n} = 0$). When τ_j is mapped on p_n (second line of Equation (16)), the first expression in the RHS depicts the temperature of processor p_n when τ_j is mapped on it but not executing at time t. The next expression in the RHS denotes the temperature of p_n when τ_j executes on it at time step t. Here, $\alpha = t - S_j$.

Complexity Analysis: The overhead of the above formulation can be analyzed in terms of the total number of variables employed considering all constraints of the problem:

- *Unique start time constraints* use $O(|V|)$ constraints and $O(|P| \times T)$ variables per constraint.
- *Dependency constraints* employ $O(|E|)$ constraints and $O(|P| \times T)$ variables per constraint.
- *Resource constraints* use $O(|P| \times T)$ constraints and $O(|V|)$ variables per constraint.
- *Execution Demand Constraints* employ $O(|P| \times |V|)$ constraints and $O(T)$ variables per constraint.
- *Processor temperature constraints* utilize $O(|P| \times T)$ constraints. The number of variables per constraint can be determined (refer to Equation (15); when $t \neq 0$ & $\lambda^t_n \neq 0$) as: For $t = 1$, the number of variables required to compute Γ^t_n is $O(|V|)$. Similarly, for $t = 2$, the number of variables is $O(|V|^2)$, and so on. For $t = T$, the number of variables is $O(|V|^T)$. Thus, the total variables per constraint becomes $O(|V| + |V|^2 + \cdots + |V|^T) = O(\frac{|V|^{T+1} - |V|}{|V|-1}) = O(|V|^T)$.

Finally, the total number of variables in all constraints of the proposed formulation becomes: $O(3 \times |V| \times |P| \times T + |E| \times |P| \times T + |V|^T \times |P| \times T) = O(|V|^T \times |P| \times T)$.

An estimate of the size of the state space associated to the problem may also be derived as follows. To determine an optimal solution, there may be a need to explore: (i) all possible orders in which the set of tasks may be considered for possible allocation, which is $O(|V|!)$, (ii) given any such order for task allocation, each task may be allocated to any processor; the number of possibility becomes $O(|P|^{|V|})$, and (iii) for each mapping of a task on a certain processor an upper bound on the number of processor temperature management schedules are obtained as $O(2^T)$. Hence, an upper bound estimate on the size of the state space may be obtained as $O(|V!| \times |P|^{|V|} \times 2^T)$.

It can be observed from the above analysis that the problem is highly exponential with respect to the number of tasks, processors, and temporal bounds in the system. Therefore, an attempt to optimally solve this scheduling problem through standard off-the-shelf solvers or exhaustive state space search is prohibitively expensive because of the inherent enormity of the state space. Hence, we have designed a list-based heuristic solution strategy for the problem at hand. This heuristic strategy attempts to deliver good and acceptable solutions while intelligently restricting the solution search space in the following ways:

- The heuristic strategy attempts to solve the overall optimization problem by partitioning it into three separate phases, namely, (i) *task prioritization*, (ii) *processor allocation*, and (iii) *temperature-aware scheduler*.
- A fast mechanism to obtain a fixed task priority order among the ready tasks that may allow the generation of low *makespan* schedules, has been designed as the first phase. This phase is a pre-processing step, which is called prior to the commencement of processor allocation and schedule generation for the given set of tasks.

- The second phase determines a suitable processor for the highest priority unallocated task, at a given intermediate point during partial schedule generation. To obtain this processor, a temperature-aware evaluation on the allocation-goodness of the selected task on all available processors, is conducted.
- For each processor on which the highest priority task may possibly be assigned, the second phase must construct a temperature-aware schedule corresponding to evaluate the goodness of this tentative task-to-processor allocation. This temperature-aware schedule generation strategy for any given task processor pair, comprises the third phase of the algorithm.

In the next section, we describe in detail this list-based heuristic algorithm called *TMDS*.

5 TMDS: THE PROPOSED SCHEDULER

The proposed list scheduling algorithm called *TMDS* takes a precedence-constrained task graph $G(V, E)$ with known thermal characteristics and a platform P, as inputs. *TMDS* consists of three phases, namely, (i) *task prioritization*, (ii) *processor allocation*, and (iii) *temperature-aware scheduler*. Algorithm 1 presents the pseudocode of *TMDS*. Steps 1-3 of function *TMDS()* comprises the *task prioritization phase*, while the *processor allocation phase* consists of steps 5–14. Step 9 of the Algorithm 1 calls function ***TETT***$(\tau_j, p_n, curT, \Gamma_n^{est})$ (***Task Execution Time and final Temperature***; refer to Algorithm 2) to generate a tentative temperature management schedule associated with the possible allocation of task τ_j on p_n. In the processor selection phase, the temperature-aware schedule returned by *TETT()* plays a significant role in the determination of the actual processor on which τ_j is allocated. We now discuss these three phases in detail.

Task prioritization: Step 2 of *TMDS* (refer to Algorithm 1) calculates two different parameters (adapted from Reference [22]), namely, (1) ***Optimistic Finish Time (OFT)*** for each task-processor pair and (2) A *Rank* value for each task.

OFT$[\tau_j, p_n]$: The value of $OFT[\tau_j, p_n]$ is a lower bound estimate of the total additional time required to complete the schedule when task τ_j is assigned on processor p_n. This duration includes the execution time of task τ_j on p_n along with the minimum time required to schedule all remaining unallocated tasks on the given platform. The sink node (τ_{exit}) has an OFT equal to its execution time, since it has no successors. The OFT for all other tasks is calculated recursively from τ_{exit} to τ_j as detailed below:

$$OFT[\tau_j, p_n] = \begin{cases} \omega_{exit,n}, & if\ \tau_j = \tau_{exit}, \\ \max\limits_{\tau_k \in succ(\tau_j)} \Big[\min\limits_{p_r \in P} \{OFT[\tau_k, p_r] + \omega_{j,n} + c_{j,k}^{n,r}\} \Big], & otherwise. \end{cases} \tag{17}$$

During the processor allocation phase, this estimated time ($OFT[\tau_j, p_n]$) value is employed to obtain a more informed decision on which processor if allocated to a given task, may possibly lead to the least makespan (refer to function $SCT[\tau_j, p_n]$; Equation (20)).

Rank$[\tau_j]$: The rank of a task τ_j ($Rank[\tau_j]$) is obtained as the average of its OFTs over all processors. Due to this ranking strategy, given a set of ready tasks, their rank values provide comparative estimates of the amounts of workload that must be completed subsequent to their execution start times. In the endeavor to achieve low makespans, a set of ready tasks is considered for processor allocation in the order of their ranks, so that tasks that need to execute comparatively larger workloads subsequent to their starts may be prioritized:

$$Rank[\tau_j] = \frac{\sum_{n=1}^{|P|} OFT[\tau_j, p_n]}{|P|}. \tag{18}$$

Step 3 initializes ready list *taskList* with the entry task τ_{entry} in it.

ALGORITHM 1: *TMDS*(G, P)

Input: *Application DAG* $G(V, E)$, *processor set* P
Output: *A temperature-aware schedule that minimizes makespan*

1 $\forall_{p_n \in P}$, Set $avail[n] = 0$;
2 Determine OFTs, $Rank$s of all tasks using Equations (17), (18);
3 Initialize ready list $taskList = \{\tau_{entry}\}$;
4 // *Let* Γ_n^{cur} *be a temporary variable that holds the temperature of processor* p_n *at any time during partial schedule generation*
5 **while** $taskList \neq \phi$ **do**
6 $\quad \tau_j$ = Extract the task with highest *Rank* from ready list $taskList$;
7 $\quad$ **for** *each processor* p_n *in* P **do**
8 $\quad\quad$ Compute $EST[\tau_j, p_n]$ using Equation (19); // *Start time of* τ_j *if selected for execution on* p_n
9 $\quad\quad$ $\Gamma_{j,n}^{est} = T_{cool}(\Gamma^a, \Gamma_n^{cur}, EST[\tau_j, p_n] - avail[n])$; // *Tentatively determine temperature of* p_n *at the instant* $EST[\tau_j, p_n]$ $(\Gamma_{j,n}^{est})$, *given: (i) its temperature at* $avail[n]$ (Γ_n^{cur}), *and (ii) that* p_n *idles in the interval* $[avail[n], EST[\tau_j, p_n]]$ $\langle EFT[\tau_j, p_n], Sch_{j,n}, \Gamma_n^{fin}\rangle = TETT(\tau_j, p_n, EST[\tau_j, p_n], \Gamma_{j,n}^{est})$; // *The TETT() function returns: (i) effective finish time* $(EFT[\tau_j, p_n])$ *of* τ_j *on* p_n, *(ii) Schedule* $(Sch_{j,n})$ *of execution and cooling intervals w.r.t. task* τ_j *on* p_n *and (iii) temperature of* p_n *at* $EFT[\tau_j, p_n]$
10 $\quad\quad$ $SCT[\tau_j, p_n] = EFT[\tau_j, p_n] + OFT[\tau_j, p_n] - \omega_{j,n}$; // *Estimate schedule completion time of* τ_j *on* p_n
11 $\quad$ Determine: $p_{n'} = \min_{p_r \in P} SCT[\tau_j, p_r]$; // *Find the processor* $p_{n'}$ *for which* $SCT[\tau_j, p_{n'}]$ *is minimum*
12 $\quad$ Assign τ_j on processor $p_{n'}$;
13 $\quad$ Set $AFT[\tau_j] = avail[n'] = EFT[\tau_j, p_{n'}]$, $\Gamma_{n'}^{cur} = \Gamma_{n'}^{fin}$; // *Update: (i) actual finish time of* τ_j $(AFT[\tau_j])$, *(ii)* $avail[n']$ *and (iii) temperature* $\Gamma_{n'}^{cur}$ *of* p'_n *at* $avail[n']$
14 $\quad$ Insert all ready to execute successor tasks of τ_j in $taskList$;

Processor allocation: The while loop (steps 5–14) generates a schedule obtained by sequentially scheduling the task with the highest *Ranked* unscheduled ready task in $taskList$. A task becomes ready when all its predecessors have already been scheduled. After selecting the task τ_j in step 6, *TMDS* determines *Effective Start Time* ($EST[\tau_j, p_n]$; refer to Equation (19)), *Task Execution Time and final Temperature* ($TETT(\tau_j, p_n, curT, \Gamma_n^{est})$: refer to the temperature-aware scheduler below) and *Schedule Completion Time* ($SCT[\tau_j, p_n]$; refer to Equation (20)) with respect to the execution of τ_j on each processor p_n (steps 7–10).

Effective Start Time (**EST**$[\tau_j, p_n]$): The EST of the entry task τ_{entry} on each processor p_n is *zero*. For all other tasks, the effective start time $EST[\tau_j, p_n]$ of τ_j on p_n is determined as

$$EST[\tau_j, p_n] = \max\left\{avail[n], \max_{\tau_i \in pred(\tau_j)}\left(AFT[\tau_i] + c_{i,j}^{m,n}\right)\right\}, \tag{19}$$

where $avail[n]$ stores the effective time at which processor p_n is available to execute a task and $AFT[\tau_i]$ records the *actual finish time* of τ_j's predecessor task τ_i. The inner $\max_{\tau_i \in pred(\tau_j)}\{\dots\}$ block determines the time at which all outputs from the predecessors of τ_j have arrived at processor p_n.

Step 9 updates $\Gamma_{j,n}^{est}$ to obtain the temperature of p_n at time $EST[\tau_j, p_n]$ (refer to Equation (8)), when the processor remains idle for a time interval $[avail[n], EST[\tau_j, p_n]]$, starting from its initial temperature Γ_n^{cur} at time $avail[n]$. Step 9 calls *TETT()* to determine Effective Finish Time $EFT[\tau_j, p_n]$, a scheduling event queue $Sch_{j,n}$ and the temperature (Γ_n^{fin}) of p_n at time $EFT[\tau_j, p_n]$.

Effective Finish Time (**EFT**$[\tau_j, p_n]$): $EFT[\tau_j, p_n]$ is the effective execution finish time of task τ_j on p_n. The effective time at which τ_j can start execution on p_n is given by $EST[\tau_j, p_n]$ (refer to Equation (19)). This time instant ($EST[\tau_j, p_n]$) is marked by a distinct temperature value of p_n, which may be the ambient temperature (if p_n has been idle for a long time) or a higher temperature

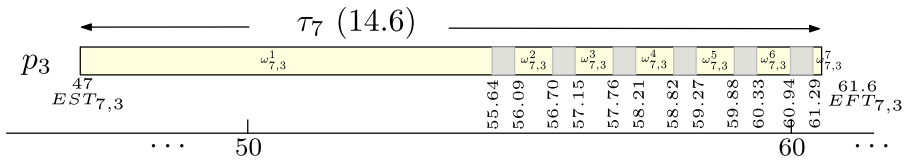


Fig. 2. $Sch_{7,3}$—schedule of task τ_7 on processor p_3.

(due to earlier task executions on p_n). Starting from this initial temperature at $EST[\tau_j, p_n]$, τ_j may be able to complete its execution demand $\omega_{j,n}$ in one continuous interval or may need one or more intervening cooling intervals (to prevent p_n from breaching the threshold temperature Γ_n^{lim}). Thus in general, the total duration over which τ_j executes on p_n is characterized by alternating sub-phases of execution and cooling, such that the aggregate length of these execution sub-phases is equal to $\omega_{j,n}$. Thus, the value of $EFT[\tau_j, p_n]$ is obtained by adding $EST[\tau_j, p_n]$ with this total duration over which τ_j executes on p_n.

Scheduling event queue ($Sch_{j,n}$): $Sch_{j,n}$ is an ordered event queue marking all the time instances corresponding to the start of execution and cool-off sub-phases within the total duration over which τ_j remains allocated on p_n for completing its execution demand ($\omega_{j,n}$). Corresponding to our running example (refer to the paragraph "Example continued"), Let $Sch_{7,3} = \langle 47.00, 55.64, 56.09, 56.70, 57.15, 57.76, 58.21, 58.82, 59.27, 59.88, 60.33, 60.94, 61.29, 61.60\rangle$ represents the event queue related to the execution of task τ_7 on processor p_3. The schedule Gantt chart produced by this event queue may be seen in Figure 4 and reproduced here in Figure 2, for convenience. It may be observed that this Gantt chart has seven alternating phases of execution and cooling. Among these, $\omega^1_{7,3} (= 55.64 - 47) = 8.64$, $\omega^2_{7,3} = \omega^3_{7,3} = \omega^4_{7,3} = \omega^5_{7,3} = \omega^6_{7,3} = 0.61$, and $\omega^7_{7,3} = 0.31$, denote the execution sub-phases. The first phase starts at $EST[\tau_7, p_3] = 47.00$. The WCET of task τ_7 on p_3 is obtained as $\omega_{7,3} = \sum_{k=1}^{7} \omega^k_{7,3} = 12$. The total duration over which τ_7 remains allocated on p_3 for completing its execution demand ($\omega_{7,3} = 12$) is 14.60. Hence, $EFT[\tau_7, p_3] = 47.00 + 14.60 = 61.60$.

Step 10 computes the Schedule Completion Time ($SCT[\tau_j, p_n]$) of task τ_j on processor p_n.

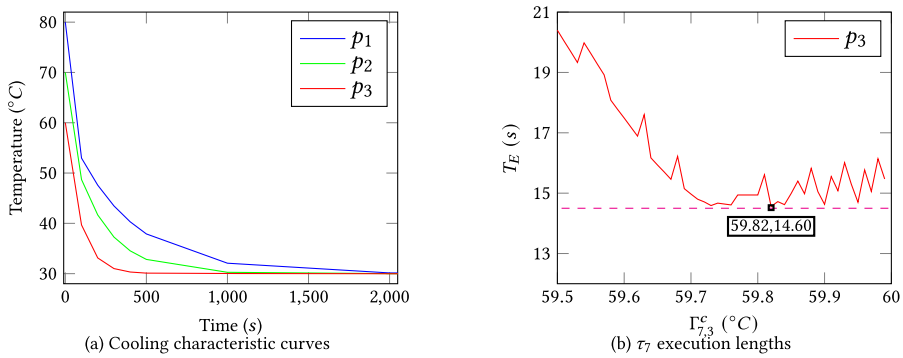
Schedule Completion Time (SCT$[\tau_j, p_n]$): Given that τ_j is allocated on p_n, $SCT[\tau_j, p_n]$ is computed as

$$SCT[\tau_j, p_n] = EFT[\tau_j, p_n] + OFT[\tau_j, p_n] - \omega_{j,n}. \tag{20}$$

The RHS of the above expression can be viewed as a combination of two terms: (i) The first term $EFT[\tau_j, p_n]$ is the effective execution finish time of task τ_j on p_n. (ii) The second term $OFT[\tau_j, p_n] - \omega_{j,n}$ essentially provides an estimate of the total time required to complete the execution of all dependant tasks of τ_j, assuming τ_j to be allocated for execution on processor p_n.

Finally in steps 11 and 12, each selected task τ_j is assigned to that processor p_n for which $SCT[\tau_j, p_n]$ is minimal. After τ_j is scheduled on p_n, *EST* and *EFT* of τ_j on p_n become the *Actual Start Time* $AST[\tau_j]$ and *Actual Finish Time* $AFT[\tau_j]$ of task τ_j. Step 13 updates the finish time of each task and captures the current temperature of p_n as well as updates the *avail* time of processor p_n. *Makespan* of the generated schedule is equal to the finish time $AFT[\tau_{exit}]$ of the sink task τ_{exit}.

Temperature-aware scheduler: The processor allocation phase calls function $TETT()$ for each possible processor on which the currently highest priority task τ_j can be assigned. *TETT()*'s objective is to generate a minimal length non-preemptive execution schedule for τ_j while ensuring that the temperature of p_n is always maintained below its stipulated threshold value. To achieve this, the temperature-aware scheduler determines the intervals within the execution schedule during which τ_j actually executes on p_n. The processor is kept idle to enable cool-off, in the remaining intervals. However, each transition from the *active* to *idle* state of a processor (or *vice versa*), has



(a) Cooling characteristic curves (b) τ_7 execution lengths

Fig. 3. (a) Cooling characteristic curves of processors w.r.t. time; (b) task execution lengths w.r.t. *cutoff* temperatures.

an overhead (OV) associated with the *saving* (or *restoration*) of the executing task's intermediate state. Over diverse embedded processing platforms having varying capabilities, this overhead may differ from about $\approx$ 10 μs up to $\approx$ 150 μs [13, 14, 35]. Throughout the remaining article, we have considered the value of OV to be = 80 μs [13].

TETT() adheres to two design principles for minimizing the total execution length of τ_j on p_n in the presence of OV, the *active-to-idle* transition overhead: (i) an interval in which task τ_j actually executes on processor p_n is never interrupted/terminated until the threshold temperature of p_n is reached, and (ii) whenever the temperature of p_n reaches its threshold value, it is switched to idle mode. Thereafter, p_n continuously cools off in this idle state for a minimum duration that either allows: (a) continuous execution of τ_j up to completion (while not violating p_n's threshold) in the execution interval, which follows this cool-off interval, or (b) cool down to the ambient room temperature. If τ_j's execution time is relatively high, or if p_n's threshold temperature is low, then multiple idle cool-off intervals may be required before τ_j is able to complete its execution. In all these intermediate cool-off intervals (except the final interval), p_n is forced to idle until ambient room temperature is reached. We now show that this obvious cool down strategy may have a very adverse effect on *TETT()*'s objective towards achieving a minimum length schedule for τ_j.

Figure 3(a) shows the cooling characteristic curves plotted using Equation (8), for three heterogeneous processors with different thermal properties, as detailed in Table 2. It can be observed from Figure 3(a) that a processor p_n must suffer an exponentially long duration of idleness if it is forced to cool down from the threshold temperature (Γ_n^{lim}) to a *cutoff* temperature, which is close to the ambient room temperature (Γ^a). However, if the *cutoff* temperature is too close to the threshold Γ_n^{lim}, then p_n cannot cool down sufficiently so that adequate execution progress can be made before temperature reaches Γ_n^{lim} again (when p_n must transit back to idle mode). Thus, we see that the total duration of execution of a task τ_j on p_n gets elongated (beyond an optimal value) both when the *cutoff* ($\Gamma_{j,n}^c$) temperature is very close to either Γ_n^{lim} or Γ^a. This indicates the existence of one or more optimal values of $\Gamma_{j,n}^c$ (which we refer to as $\Gamma_{j,n}^{oc}$; $\Gamma_n^{lim} > \Gamma_{j,n}^{oc} > \Gamma^a$) for which the total execution duration of τ_j on p_n gets minimized.

As discussed above, the temperature of a processor p_n (on which task τ_j is considered for assignment) at the beginning of τ_j's execution (at the instant $EST[\tau_j, p_n]$) is obtained as Γ_n^{est} in step 9 of Algorithm 1. Now, the total execution length of τ_j on p_n ($T_E(\tau_j, p_n)$) can be expressed as

$$T_E(\tau_j, p_n) = MDE\left(\tau_j, p_n, \Gamma_{j,n}^{est}\right) + \lfloor I \rfloor \times \left[coolT\left(\Gamma_{j,n}^c, \Gamma_n^{lim}\right) + MDE\left(\tau_j, p_n, \Gamma_{j,n}^c\right)\right] + remET + coolT\left(\Gamma_{j,n}^{cur}, \Gamma_n^{lim}\right) + 2\lceil I \rceil \times OV. \quad (21)$$

Table 2. Thermal Parameters of Processors [17, 20]

	f	C_0	C_1	C_2	R	C	Γ^{lim}	Γ^{cur}	Γ^{a}
p_1	2.6	2.332	13.1568	0.1754	0.68	380	80 °C	30 °C	25 °C
p_2	3.4	2.138	5.0187	0.1942	0.487	295	70 °C	50 °C	25 °C
p_3	3.0	4.556	15.6262	0.1942	0.238	320	60 °C	40 °C	25 °C

The above equation can be explained by describing the important terms in its RHS. Starting with the initial temperature $\Gamma^{est}_{j,n}$, $MDE(\tau_j, p_n, \Gamma^{est}_{j,n})$ (***MDE: Maximum Duration of continuous Execution***) represents the execution progress that τ_j makes by continuously running on p_n until the temperature reaches p_n's threshold value Γ^{lim}_n. The function $MDE()$ can be derived from Equation (7) as

$$MDE(\tau_j, p_n, \Gamma^{ini}) = \begin{cases} \infty, & if\, \Gamma^{ss}_{j,n} \leqslant \Gamma^{lim}_n, \\ \frac{-1}{\mathcal{B}_n} \times log(\frac{\Gamma^{lim}_n - \Gamma^{ss}_{j,n}}{\Gamma^{ini} - \Gamma^{ss}_{j,n}}), & otherwise. \end{cases} \tag{22}$$

Next, $coolT(\Gamma^{c}_{j,n}, \Gamma^{lim}_n)$ denotes the idle interval that is necessary for p_n to cool off from Γ^{lim}_n to $\Gamma^{c}_{j,n}$. By replacing Γ^{fin} and Γ^{ini} by $\Gamma^{c}_{j,n}$ and Γ^{lim}_n, a generalised definition of $coolT()$ may also be obtained from Equation (7), as

$$coolT(\Gamma^{fin}, \Gamma^{ini}) = \frac{-1}{\mathcal{B}_n} \times log\left(\frac{\Gamma^{fin} - \Gamma^{a}}{\Gamma^{ini} - \Gamma^{a}}\right). \tag{23}$$

The term I denotes the number of cooling intervals needed for τ_j to complete execution on p_n:

$$I = \frac{\omega_{j,n} - MDE(\tau_j, p_n, \Gamma^{est}_{j,n})}{MDE(\tau_j, p_n, \Gamma^{c}_{j,n})}. \tag{24}$$

$remET$ represents the last remaining execution chunk of τ_j on p_n before completion, and is obtained as

$$remET = \omega_{j,n} - MDE\left(\tau_j, p_n, \Gamma^{est}_{j,n}\right) - \lfloor I \rfloor \times MDE\left(\tau_j, p_n, \Gamma^{c}_{j,n}\right). \tag{25}$$

The maximum temperature ($\Gamma^{cur}_{j,n}$) necessary to continuously execute τ_j for its last remaining $remET$ time units, while not violating the threshold temperature Γ^{lim}_n, is defined as

$$\Gamma^{cur}_{j,n} = \Gamma^{ss}_{j,n} + \frac{\Gamma^{lim}_n - \Gamma^{ss}_{j,n}}{e^{-\mathcal{B}_n \times remET}}. \tag{26}$$

Finally, OV denotes the *active* to *idle* (or *idle* to *active*) transition overhead associated with the *saving* (or *restoration*) of task τ_j intermediate state. As discussed above, the value of OV has been assumed to be 80 μs.

Example 1: Let us consider the 10 task DAG in Figure 1(a). The DAG is to be executed on three heterogeneous processors. The WCETs of each task on the three processors are given in Figure 1(b). The processors' thermal parameters have been adopted from References [17, 20] and listed in Table 2. Let the *steady-state temperatures* corresponding to these tasks for the given processors be as listed in Figure 1(b). Given OV, the optimal *cutoff* temperature of τ_j on p_n ($\Gamma^{oc}_{j,n}$) corresponds to that value of $\Gamma^{c}_{j,n}$ for which the lowest minima (having the longest *cutoff* interval) of function $T_E(\tau_j, p_n)$ (refer to Equation (21)) is obtained, as $\Gamma^{c}_{j,n}$ is varied in the range $[\Gamma^{a}, \Gamma^{lim}_n]$. By putting $\Gamma^{oc}_{j,n}$ as the first argument of $coolT()$ (refer to Equation (23)), we get the optimal *cool-off* interval size ($I^{oc}_{j,n}$).

Table 3. Schedule Produced by *TMDS* in Each Iteration for DAG in Figure 1

taskList	τ_j	R	OFT			EST			EFT			SCT			Allocation		
			p_1	p_2	p_3	p_1	p_2	p_3	p_1	p_2	p_3	p_1	p_2	p_3	$AFT[\tau_j]$	p_n	Γ_n^{cur}
τ_1	τ_1	44.8	46.5	44.0	44.0	0.0	0.0	0.0	16.0	15.0	14.0	62.50	59.0	58.0	14.0	p_3	46.4 °C
τ_5,τ_2,τ_4,τ_3,τ_6	τ_5	28.4	32.0	29.33	24.0	17.0	16.0	14.0	36.0	34.0	34.0	68.0	63.33	58.0	34.0	p_3	53.4 °C
τ_2,τ_4,τ_3,τ_6	τ_2	28.2	31.5	29.0	24.0	16.50	15.67	34.0	32.50	32.67	52.0	64.0	61.67	76.0	32.67	p_2	52.1 °C
τ_4,τ_3,τ_6	τ_4	26.5	28.5	27.0	24.0	17.50	32.67	34.0	36.50	42.67	51.0	65.0	69.67	75.0	36.50	p_1	33.3 °C
τ_3,τ_6,τ_9	τ_3	25.1	29.5	26.67	19.0	36.50	32.67	34.0	54.50	48.67	47.0	84.0	75.33	66.0	47.0	p_3	57.3 °C
τ_6,τ_9,τ_7	τ_6	23.5	27.5	22.0	21.0	36.50	32.67	47.0	51.50	48.67	63.62	79.0	70.67	84.62	48.67	p_2	55.1 °C
τ_9,τ_7,τ_8	τ_9	11.3	15.0	12.0	7.0	41.67	48.67	47.0	61.67	67.67	69.78	76.67	79.67	76.78	61.67	p_1	36.6 °C
τ_7,τ_8	τ_7	11.3	15.0	12.0	7.0	61.67	48.67	**47.0**	81.67	66.67	61.60	96.67	78.67	68.60	**61.60**	p_3	60.0 °C
τ_8	τ_8	11.3	15.0	12.0	7.0	61.67	48.67	61.84	77.67	60.67	85.16	92.67	72.67	92.16	60.67	p_2	57.1 °C
τ_{10}	τ_{10}	0.0	0.0	0.0	0.0	65.67	67.67	64.67	80.67	79.67	73.62	80.67	79.67	73.62	73.62	p_3	60.0 °C

ALGORITHM 2: $TETT(\tau_j, p_n, curT, \Gamma_{j,n}^{est})$

Input: *Task* τ_j, *processor* p_n, *time* $curT$, *temperature* of p_n at $EFT[\tau_j, p_n]$ $(\Gamma_{j,n}^{est})$
Output: *Schedule Sch and effective finish time EFT of a task* τ_j *on* p_n *and temperature of* p_n *at EFT*

$allowET = MDE(\tau_j, p_n, \Gamma_{j,n}^{est})$ refer to Equation (22);
if $\omega_{j,n} \leqslant allowET$ **then**
  return $\langle Sch_{j,n}, curT + \omega_{j,n}, T_{heat}(\Gamma_{j,n}^{ss}, \Gamma^0, \omega_{j,n})\rangle$;
else
  $remET = \omega_{j,n} - allowET$;
  $curT = curT + allowET$;
$idleT = coolT(\Gamma_{j,n}^{c}, \Gamma_n^{lim})$ refer to Equation (23);
$allowET = MDE(\tau_j, p_n, \Gamma_{j,n}^{c})$ refer to Equation (22);
while $remET > 0$ **do**
  if $remET \geqslant allowET$ **then**
    // p_n *should idle for an interval* $[curT, curT + idleT + 2 \times OV]$
    $curT = curT + idleT + allowET + 2 \times OV$;
    // τ_j *should execute for an interval* $[curT - allowET, curT]$
  else
    Calculate $\Gamma_{j,n}^{cur}$ (using Equation (26)), the max temperature necessary to continuously execute τ_j for its last remaining $remET$ time units, while not breaching Γ_n^{lim};
    $idleT = coolT(\Gamma_{j,n}^{cur}, \Gamma_n^{lim})$ refer to Equation (23);
    $curT = curT + idelT + remET + 2 \times OV$;
  $remET = remET - allowET$;
return $\langle curT, Sch_{j,n}, \Gamma_n^{lim}\rangle$;

Figure 3(b) depicts τ_7's execution lengths on processors p_3, as the $\Gamma_{j,n}^{c}$ is progressively lowered from $\Gamma_3^{lim} = 60$ °C. The plots in Figure 3(b) validate the existence of an optimal cutoff temperature (for each distinct task-processor pair) for which the total execution duration T_E, is minimal. For example, $\Gamma_{7,3}^{oc}$, $I_{7,3}^{oc}$, and corresponding T_E's for τ_7 are obtained as $(\langle\Gamma_{7,3}^{oc}, I_{7,3}^{oc}, T_E\rangle)$; $\langle 59.82\ ^\circ C, 0.45s, 14.60s\rangle$ (indicated by black dot in the figure).

TETT(): The pseudocode of function *TETT()* is presented in Algorithm 2. The objective of *TETT()* is to generate an execution/idling schedule for a task τ_j on p_n, while ensuring that the temperature of p_n remains below the specified threshold value. *TETT()* returns this schedule along with the

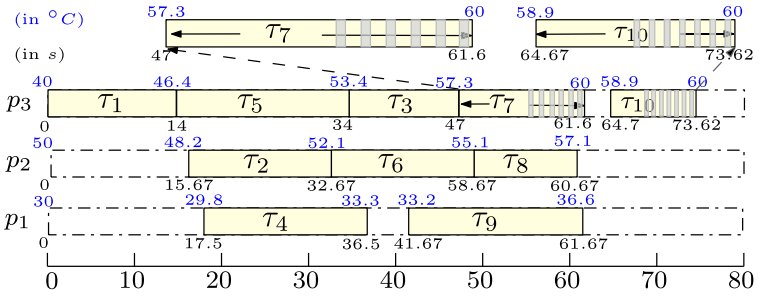


Fig. 4. Gantt chart of the schedule generated by *TMDS* (*makespan* = 73.62*s*) for the DAG in Figure 1(a).

effective finish time $EFT[\tau_j, p_n]$ and the processor temperature at $EFT[\tau_j, p_n]$. Given the initial temperature Γ_n^{est} of p_n, step 1 of Algorithm 2 determines the Maximum Duration of continuous Execution ($MDE()$; using Equation (22)) of τ_j before p_n's temperature reaches Γ_n^{lim} (and stores it in *allowET*). When $\omega_{j,n} \leqslant allowET$ is true (step 2), function *TETT()* can directly return (step 3) with the *schedule* $Sch_{j,n}$ being a single continuous execution interval (with no idling), $EFT[\tau_j, p_n]$ being $EST[\tau_j, p_n]+\omega_{j,n}$ and the *temperature* of p_n at time $EFT[\tau_j, p_n]$ calculated using $T_{heat}(\Gamma_{j,n}^{ss}, \Gamma^0, \omega_{j,n})$. Otherwise (steps 4–6), temperature of p_n reaches Γ_n^{lim} after *allowET* units of continuous execution. Steps 5 and 6, respectively, update the current values of τ_j's remaining execution requirement (*remET*) as well as the time up to which it must remain assigned on p_n.

Now, to complete *remET* units of execution *TETT()*, must enter into a repetitive loop (steps 9–18). At each iteration of the loop, depending on the value of *remET* (step 10), p_n must cool down either up to the optimal cutoff temperature $\Gamma_{j,n}^{oc}$ (step 12) or to a specific temperature from which τ_j can be continuously executed to completion while not breaching the threshold temperature Γ_n^{lim} (step 15). Step 7 calculates the interval that is necessary for p_n to cool off from Γ_n^{lim} to $\Gamma_{j,n}^{c}$, while step 8 determines the continuous execution duration that will result in p_n's temperature to increase from $\Gamma_{j,n}^{c}$ to Γ_n^{lim}. The remaining steps of the algorithm are self-explanatory. In step 19, *TETT()* finally returns with a schedule $Sch_{j,n}$ (that includes all the idle and execution intervals of τ_j), effective finish time $EFT[\tau_j, p_n]$ and the temperature (Γ_n^{lim}) of p_n at time $EFT[\tau_j, p_n]$.

Example Continued: Using the same example system (refer to Example 1), let us assume that the communication links between each pair of processors have the following bandwidths: $b_{1,2} = b_{2,1} = 1$, $b_{2,3} = b_{3,2} = 3$, $b_{3,1} = b_{1,3} = 2$, and $b_{1,1} = b_{2,2} = b_{3,3} = \infty$. The rows in Table 3 show the values of important terms, at each iteration during partial schedule generation by *TMDS()* (*while* loop steps 5–14, Algorithm 1). The Gantt chart in Figure 4 depicts the schedule generated by *TMDS()* (*makespan* 73.62 s).

In the Gantt chart, the light-yellow-coloured intervals denote phases when a task actually executes on a processor. However, the gray-coloured intervals denote phases when a processor idles with a certain task assigned on it. For example, when the task τ_7 is allocated on processor p_3 at time $EST[\tau_{7,3}] = 47$ s (refer to Table 3), the operating temperature of p_3 is $\Gamma_{7,3}^{est} = 57.3$ °C. At this time, p_3 is allowed a continuous execution interval of 8.64 s before reaching p_3's threshold ($\Gamma_3^{lim} = 60$ °C). To complete τ_7's remaining execute of 3.36 s ($= 12 - 8.64$ s), p_3 must cool down either up to the optimal cutoff temperature $\Gamma_{7,3}^{oc} = 59.70$ °C (with interval size $I_{7,3}^{oc} = 0.45$ s) or to a specific temperature from which τ_7 can be continuously executed to completion, while not breaching the threshold temperature. To do so, processor p_3 remains idle in the following six intervals ⟨(*start time, start temperature*), (*end time, end temperature*)⟩: ⟨(55.64 s, 60 °C), (56.09 s, 59.70 °C)⟩, ⟨(56.7 s, 60 °C), (57.15 s, 59.70 °C)⟩, ⟨(57.76 s, 60 °C), (58.21 s, 59.70 °C)⟩, ⟨(58.82 s, 60 °C), (59.27 s, 59.70 °C)⟩, ⟨(59.88 s, 60 °C), (60.33 s, 59.70 °C)⟩, and ⟨(60.94 s, 60 °C), (61.29 s, 59.89 °C)⟩.

Therefore, the total allocation duration of task τ_7 on p_3 is 14.6 s. This primarily includes τ_7's execution duration (12 s) and p_3's idle intervals ($5 \times .45$ s + 0.35 s = 2.6 s). It may be noted from Table 3 that the start time of τ_7 on p_3 is 47 s and its finish time is 61.6 s.

Complexity Analysis: In terms of time complexity, *TMDS* requires the computation of an *OFT* table that is $O(|P|(|V|+|E|))$, and the allocation of tasks-to-processors require an overhead of $O(|E|\times|P|)$. Thus, the overall complexity of *TMDS* can be expressed as: $O(|P|(|V|+|E|)+|E|\times|P|) \approx O(|E| \times |P|)$. The time complexity of *TMDS* has been found to be similar to the majority of the mentioned prior works [1, 7, 9, 30, 37], in spite of its temperature-aware nature.

6 PERFORMANCE EVALUATION

In this section, we experimentally assess the performance of *TMDS* against *five* existing strategies as discussed in Section 6.1. Next, we present the experimental setup and the performance metrics in the following subsections, followed by detailed experimental results.

6.1 Comparison with Related Works

To the best of our knowledge, the work proposed herein (*TMDS*) is the first one to deal with the temperature-aware scheduling of precedence-constrained DAG-structured applications on distributed heterogeneous processing platforms. That is, none of the well-known DAG scheduling strategies for heterogeneous platforms such as *HEFT* [30], *PEFT* [1], *PPTS* [7], *PSLS* [37], and *ODS* [9] are temperature-aware. Temperature-aware counterparts for any of these strategies are also not available in the literature. In this regard, it may be highlighted that the temperature-awareness mechanism presented in this article is generic and adaptable. Hence, the proposed mechanism can be easily employed to extend the DAG scheduling strategies mentioned above and obtained their temperature versions as follows. In general, all these strategies have a two-level design. The first level among them is concerned with task priority generation, while the second level deals with the allocation of processors-to-tasks in the order prescribed by the first level. While considering a processor for a task, the temperature-awareness mechanism (refer to *TETT()*) determines and takes into account the time required to complete the task on the given processor while never breaching the temperature cap associated with the processor. Following this approach, we have implemented temperature-aware versions (namely, *THEFT*, *TPEFT*, *TPPTS*, *TPSLS,* and *TODS*) of the prominent existing algorithms *HEFT*, *PEFT*, *PPTS*, *PSLS,* and *ODS*.

6.2 Experimental Setup

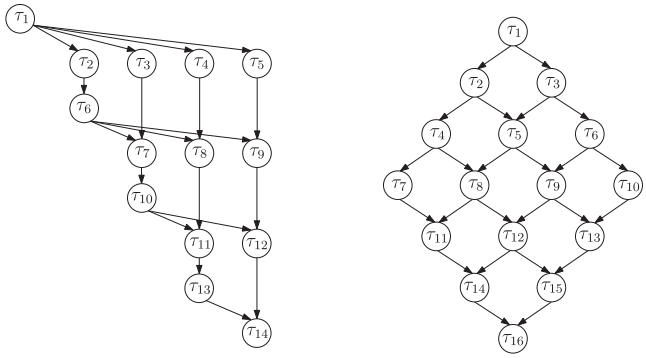
The experimental evaluation has been conducted using two real-world benchmark DAGs: Gaussian Elimination [23] and Laplace [19]. The structural representations of these task graphs are depicted in Figure 5.

Gaussian Elimination is a method used to solve a system of linear equations. The DAG structure of this graph is characterized by *matrix-size* (ν). A Gaussian Elimination DAG has $(\nu^2+\nu-2)/2$ tasks and $(\nu^2-\nu-1)$ edges. For example, Figure 5(a) depicts the DAG structure of Gaussian Elimination for *matrix-size* $\nu = 5$. The DAG has 14 tasks and 19 edges.

The **Laplace** graph is derived from the Laplace equation solver algorithm. The size of this task graph is influenced by the size of the input parameter φ. The total number of tasks and edges in a Laplace DAG is equal to φ^2 and $2\varphi(\varphi-1)$. For example, Figure 5(b) shows a Laplace task graph for $\varphi = 4$. The task graph consists of 16 tasks and 24 edges.

Data Generation Framework: We have generated an exhaustive set of random input test cases by carefully varying the following set of parameters to conduct our experiments.

(1) **Task graph size:** For Gaussian Elimination, the *matrix-sizes* $\nu = \{8, 12, 15, 18, 20\}$, respectively, produces task graphs having $|V| = \{35, 77, 119, 170, 209\}$ tasks and $|E| = \{55, 131, 209, 305,$



(a) Gaussian Elimination (b) Laplace

Fig. 5. Benchmark task graphs.

379} edges. Similarly for Laplace, $\varphi = \{6, 9, 11, 13, 14\}$ results in task graphs having $|V| = \{36, 81, 121, 169, 196\}$ and $|E| = \{60, 144, 220, 312, 364\}$.

(2) **Number of processors:** $|P| = \{4, 8, 16, 32\}$.

(3) **Task execution times:** Our experiments used a three-step approach to produce the execution time of each task on different heterogeneous processors. First, we fixed the average execution time ($\overline{\omega_{DAG}}$) of all tasks in the graph. $\overline{\omega_{DAG}} = \{10, 15, 20, 25\}$ has been considered in our experiment. Given $\overline{\omega_{DAG}}$, the average execution time ($\overline{\omega_j}$) of task τ_j over all processors is generated. Values of $\overline{\omega_j}$ for different tasks are generated from normal distributions ($\mathcal{N}(\mu, \sigma)$) with mean $\mu = \overline{\omega_{DAG}}$ and various standard deviation values $\sigma = \{1, 2, 3\}$. Finally, we obtain the execution times of task τ_j on each processor p_n at $\omega_{j,n} \sim \mathcal{N}(\overline{\omega_j}, \overline{\omega_j} \times \beta)$. Here, $\beta = \{0.1, 0.25, 0.5, 0.75, 1\}$ is the heterogeneity factor that indicates the skewness among execution times of a task on different processors.

(4) **Communication-to-Computation Ratio (CCR):** This parameter determines the ratio of relative overhead between message transmission and task execution. A higher CCR indicates that the system spends relatively more time in transmission of data between processors than task execution. Values of CCR chosen in this work are: $CCR = \{0.1, 0.25.0.5, 0.75, 1\}$. The mean inter-task message size ($\overline{data_{DAG}}$; in bytes) for a task graph is given by: $\overline{data_{DAG}} = CCR \times \overline{\omega_{DAG}} \times \overline{B}$. Here, $\overline{B}$ ($= \frac{1}{|P| \times (|P|-1)/2} \sum b_{m,n}$ $(1 \leqslant m \leqslant |P|; 1 \leqslant n < m)$) is the average communication bandwidth randomly sampled from $\overline{B} = \{1\ Gbps, 5\ Gbps\}$. An element $b_{m,n} \in B$ denotes the actual bandwidth of each link between p_m, p_n and is sampled from a normal distribution such that $b_{m,n} \sim \mathcal{N}(\overline{B}, 0.2 \times \overline{B})$. These $b_{m,n}$ values are further scaled appropriately such that $\sum_{m=1}^{|P|} \sum_{n=1}^{m-1} b_{m,n}$ becomes $|P| \times (|P| - 1)/2 \times \overline{B}$. The output message size ($data_{i,j}$) for each edge (τ_i, τ_j) in the task graph is sampled from a normal distribution $data_{i,j} \sim \mathcal{N}(\overline{data_{DAG}}, 0.2 \times \overline{data_{DAG}})$ and then scaled such that $\sum data_{i,j}$ over all edges in the task graph equals $|E| \times \overline{data_{DAG}}$.

(5) **Temperature parameters:** Similar to Reference [17], the steady-state temperature ($\Gamma^{ss}_{j,n}$) of a task τ_j on p_n is proportional to its intensity. A more intensive task will cause the processor to heat up to higher temperatures. The value of $\Gamma^{ss}_{j,n}$ for each task-processor pair is generated by sampling uniformly from the range [40 °C, 120 °C]. Similarly, the temperature threshold (Γ^{lim}_n) for each processor p_n is picked uniformly from the range [60 °C, 100 °C]. The thermal constant ($\mathcal{B}_n$) determines the thermal characteristics of the processor. A processor with higher $\mathcal{B}_n$ tends to quickly heat (cool) up (down). Values of $\mathcal{B}_n$ are sampled from a normal distribution with $\mu = 0.008$ and $\sigma = 0.004$. The ambient temperature (Γ^a) is the temperature of the environment where the system operates and is fixed to 25 °C for all our experiments

Table 4. Pair-wise Comparison of Algorithms Using Gaussian Elimination

		TODS	*TPSLS*	*TPPTS*	*TPEFT*	*THEFT*
TMDS	better	67.8%	66.7%	62.2%	54.0%	65.6%
	equal	1.3%	6.5%	1.3%	14.2%	1.3%
	worse	30.9%	26.8%	36.5%	31.8%	33.1%
THEFT	better	57.3%	43.7%	40.5%	39.3%	
	equal	1.5%	1.1%	1.7%	1.6%	
	worse	41.2%	55.2%	57.8%	59.1%	
TPEFT	better	64.1%	56.3%	54.9%		
	equal	2.1%	12.5%	1.8%		
	worse	33.8%	31.2%	43.3%		
TPPTS	better	60.3%	52.1%			
	equal	0.8%	1.4%			
	worse	38.9%	46.5%			
TPSLS	better	57.7%				
	equal	1.5%				
	worse	40.8%				

The simulation framework is written in C and is executed on a system consisting the following configuration: (i) Intel Core i7-8550U CPU @ 1.8 GHz x 8, (iii) 8 GB Memory, and (ii) Ubuntu 20.04 LTS OS.

6.3 Performance Metrics

Performance of *TMDS* has been evaluated using the following three metrics:

(1) **Schedule Length Ratio (SLR):** The *makespan*/*schedule length* is the most commonly used performance measure of a DAG scheduling algorithm. Since, a large set of DAGs with various input parameters are used, we used a normalized schedule length measure named *Schedule Length Ratio (SLR)*. This metric compares the performance of the proposed temperature-aware *makespan* minimizing scheduler *TMDS*, against *THEFT*, *TPEFT*, *TPPTS*, *TPSLS*, and *TODS*. Formally, *SLR* is determined as

$$SLR = \frac{X_{ms}}{\sum_{\tau_j \in CP_{AVG}} \overline{\omega_j}}, \tag{27}$$

where X_{ms} symbolizes the *makespan* delivered by a scheduler like *TMDS*, *THEFT*, *TPEFT*, *TPPTS*, *TPSLS*, and *TODS*. Considering execution time of each task to be its average value over all processors, the denominator depicts sum of the execution times of all tasks in the critical path (CP_{AVG}) of the DAG. It may be observed that lower the achieved *SLR*, better is the performance of the scheduling algorithm.

(2) **Number of Improved Makespans:** We used this metric to pair-wise compare the *makespan* of two scheduling algorithms in a tabular format. In the table, we show the percentages of cases for which one strategy has performed better, similar or worse compared to the other.

(3) **Run-time:** This metric determines the average run-time taken by a scheduling algorithm for data sets generated using a fixed set of input parameter values.

6.4 Experimental Results

In this subsection, we present detailed experimental results using two benchmarks: Gaussian Elimination [23] and Laplace [19].

Experiment 1: Table 4 shows pair-wise *makespan* comparisons of *TMDS* with the algorithms *THEFT*, *TPEFT*, *TPPTS*, *TPSLS*, and *TODS*. Specifically, the result corresponding to the (*row i*,

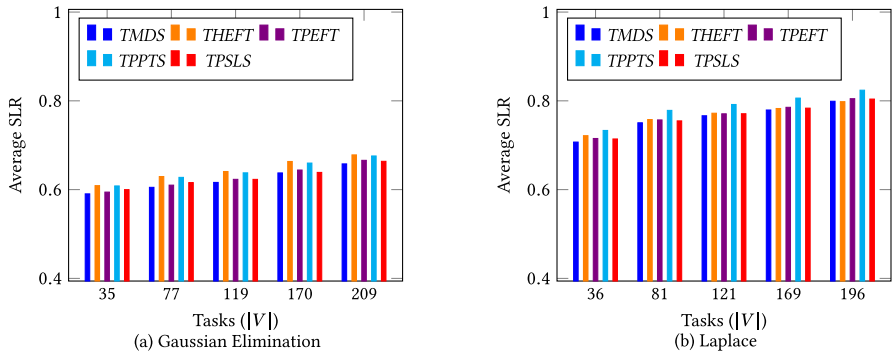


Fig. 6. Schedule length ratios for varying number of tasks.

column j)th-entry in the table depicts the percentages of test cases for which the algorithm corresponding to the *i*th row performs better, equal or worse than the algorithm in column *j*. A total of 100, 000 test cases using Gaussian Elimination task graphs have been considered by varying values of #tasks ($|V|$) and #processors ($|P|$), heterogeneity (β) and **Communication-to-Computation Ratios (*CCR*)** for each pair of algorithms. For example, the $(1, 1)$th- entry in Table 4 shows that *TMDS* performs better, equal, and worse in 66.7%, 6.5%, and 26.8% test cases, respectively, compared to *TPSLS*. The results for Laplace have not been presented as they exhibit trends very similar to Gaussian Elimination.

Experiment 2: This experiment measures the ***Schedule Length Ratios* (*SLR*)** of *TMDS*, *THEFT*, *TPEFT*, *TPPTS*, and *TPSLS* for varying values of #tasks ($|V|$). We have also conducted the experiments for varying #processors ($|P|$), heterogeneity (β), and CCR. However, the results for varying $|P|$, β, and *CCR* have not been presented here as the result trends are very similar. Each data point in the experiments below are obtained as the average over solutions generated with 200 different task graph data corresponding to a fixed set of parameter values. Obtained results for both Gaussian Elimination and Laplace are presented in Figure 6.

Figure 6 shows average SLR values for the Gaussian Elimination and Laplace task graphs, as the number of tasks varies between $\approx$ 30 and $\approx$ 200. Values of the parameters *CCR*, $|P|$, and β have been fixed at 0.5, 8, and 0.75, respectively. For Gaussian Elimination, *TMDS* is seen to deliver better results than *THEFT*, *TPEFT*, *TPPTS*, and *TPSLS*, in all cases. As an example of *TMDS's* performance, in Figure 6(a) (Gaussian Elimination) for $|V| = 119$, the average schedule lengths of *TMDS* is lower than *THEFT*, *TPEFT*, *TPPTS*, and *TPSLS* by approximately 3.99%, 1.12%, 3.50%, and 1.11%. For Laplace (Figure 6(b)), performs of *TMDS* may be seen to be slightly better or comparable to the other algorithms. As is obvious, the *makespans* (SLR values) increase as workloads become higher with increase in the number of tasks.

Experiment 3: This experiment measures the *run-time*s of *TMDS*, *THEFT*, *TPEFT*, *TPPTS*, and *TPSLS* for varying values of #tasks ($|V|$). Obtained results for Laplace are detailed in Table 5. The parameters $|P|$, *CCR*, and β are set to 8, 0.5, and 0.75, respectively. It is observed that the *run-time*s of all the schedulers strictly increase with the number of tasks. The *run-time*s of *TMDS* is upper bounded by $\approx$ 0.94 ms for the considered test cases.

7 CASE STUDY

To demonstrate the practical applicability of *TMDS* in real-world settings, we present a case study using an **Adaptive Cruise Controller (ACC)** in automotive systems. ACC automatically preserves a safe distance between two cars [23]. Figure 7(a) illustrates the ACC block diagram adapted

Table 5. Average Run-times (in ms) of Different Algorithms Using Laplace

τ_j	*TMDS*	*THEFT*	*TPEFT*	*TPPTS*	*TPSLS*
36	0.10	0.06	0.12	0.12	0.17
81	0.27	0.15	0.29	0.30	0.51
121	0.50	0.29	0.51	0.52	1.03
169	0.71	0.43	0.72	0.75	1.58
196	0.94	0.56	0.95	0.96	2.10

Table 6. ACC: $\omega_{j,n}$ (in s) and $\Gamma^{ss}_{j,n}$ (°C) of τ_j on Two Processors

		τ_1	τ_2	τ_3	τ_4	τ_5	τ_6	τ_7	τ_8	τ_9	τ_{10}	τ_{11}	τ_{12}	τ_{13}	τ_{14}	τ_{15}	τ_{16}	τ_{17}	τ_{18}	τ_{19}	τ_{20}
ω	p_1	0.9	0.6	0.6	1	0.8	0.9	1	0.9	0.9	1	1	0.8	0.9	0.6	1	1	1	1	0.9	0.9
	p_2	0.8	0.9	0.8	0.8	0.7	1	0.9	0.8	0.8	0.8	0.9	0.9	0.7	0.5	1	0.9	0.9	0.9	0.6	0.8
Γ^{ss}	p_1	98	97	99	96	98	99	91	95	93	97	94	99	97	93	94	92	95	96	94	97
	p_2	84	84	88	74	86	86	83	82	86	81	75	78	77	86	81	83	84	81	89	93

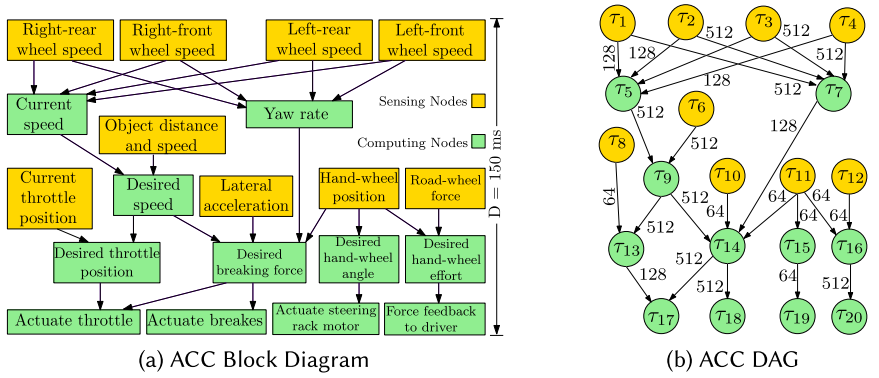


Fig. 7. Adaptive Cruise Controller (ACC): (a) Block Diagram, (b) DAG.

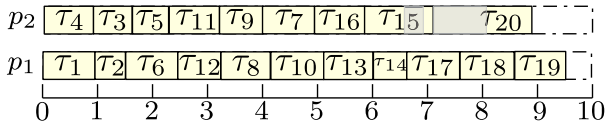


Fig. 8. Gantt chart of the ACC schedule generated by *TMDS* (*makespan* = 9.5 s) for the DAG in Figure 7(b).

from Reference [12], and Figure 7(b) displays its related DAG structure. This ACC application consists of 20 executable tasks that need to be scheduled on a platform with two processors p_1 and p_2 having the thermal properties $\langle \mathcal{B}_n, \Gamma_n^{lim}, \Gamma_n^{cur} \rangle$: $\langle 0.02267, 70, 45 \rangle$ and $\langle 0.0313, 60, 55 \rangle$, respectively.

Table 6 lists the WCETs and the steady-state temperature of the ACC task graph on the three heterogeneous processors. The communication bandwidths between the processors are considered to be 500 KB/s. Employing *TMDS*, we generate a schedule for the ACC application and the respective Gantt chart is presented in Figure 8. It may be observed that, *TMDS* is able to deliver a schedule with *makespan* (9.5 s) while satisfying the given thermal constraints.

8 CONCLUSION

This article presented *TMDS*, a low-overhead temperature aware list scheduling algorithm for DAG-structured applications on distributed heterogeneous systems. The objective of *TMDS* is to generate a *makespan* minimizing schedule while satisfying (i) execution and communication demands of application tasks, (ii) processor capacity and communication bandwidth constraints of the platform, and (iii) temperature threshold bound associated with all processors, over the entire schedule. The proposed temperature management strategy is a generic approach that can be easily used to adapt existing *makespan* minimizing DAG schedulers (for example, *THEFT*, *TPEFT*, etc.), so that the delivered schedules never violate threshold temperature bounds of processors. This generic approach is important, because as shown in Table 4, although *TMDS* is better or comparable in performance to the other state-of-the-art algorithms in a majority of considered test cases, there still are a significant number of test case scenarios in which one or more existing algorithms deliver slightly better results than *TMDS*. Our generic temperature management scheme can be used to employ *TMDS* or temperature-aware versions of any of the other algorithms as needed, in a given system scenario. Finally, the practical applicability of the proposed scheme has been additionally exhibited using a real-world case study with an adaptive cruise controller from the automotive systems domain.

REFERENCES

[1] H. Arabnejad and J. G. Barbosa. 2014. List scheduling algorithm for heterogeneous systems by an optimistic cost table. *IEEE Trans. Parallel Distrib. Syst.* 25, 3 (2014), 682–694.

[2] Evripidis Bampis, Dimitrios Letsios, Giorgio Lucarelli, Evangelos Markakis, and Ioannis Milis. 2013. On multiprocessor temperature-aware scheduling problems. *J. Sched.* 16, 5 (2013), 529–538.

[3] Martin Birks and Stanley P. Y. Fung. 2014. Temperature aware online scheduling for throughput maximisation: The effect of the cooling factor. *Sustain. Comput.: Inform. Syst.s* 4, 3 (2014), 151–159.

[4] Luiz F. Bittencourt, Rizos Sakellariou, and Edmundo R. M. Madeira. 2010. DAG scheduling using a lookahead variant of the heterogeneous earliest finish time algorithm. In *Proceedings of the 18th Euromicro Conference on Parallel, Distributed, and Network-Based Processing (PDP'10)*. IEEE, 27–34.

[5] Louis-Claude Canon, Emmanuel Jeannot, Rizos Sakellariou, and Wei Zheng. 2008. Comparative evaluation of the robustness of dag scheduling heuristics. In *Grid Computing*. Springer, 73–84.

[6] Mohammad I. Daoud and Nawwaf Kharma. 2008. A high performance algorithm for static task scheduling in heterogeneous distributed computing systems. *J. Parallel Distrib. Comput.* 68, 4 (2008), 399–409.

[7] Hamza Djigal, Jun Feng, and Jiamin Lu. 2019. Task scheduling for heterogeneous computing using a predict cost matrix. In *Proceedings of the 48th International Conference on Parallel Processing: Workshops*. 1–10.

[8] Huang Huang, Vivek Chaturvedi, Gang Quan, Jeffrey Fan, and Meikang Qiu. 2014. Throughput maximization for periodic real-time systems under the maximal temperature constraint. *ACM Trans. Embed. Comput. Syst.* 13, 2s (2014), 1–22.

[9] Jing Huang, Renfa Li, Xun Jiao, Yu Jiang, and Wanli Chang. 2020. Dynamic DAG scheduling on multiprocessor systems: Reliability, energy, and makespan. *IEEE Trans. Comput.-Aided Design Integr. Circ. Syst.* 39, 11 (2020), 3336–3347. https://doi.org/10.1109/TCAD.2020.3013045

[10] E. Ilavarasan and P. Thambidurai. 2007. Low complexity performance effective task scheduling algorithm for heterogeneous computing environments. *J. Comput. Sci.* 3, 2 (2007), 94–103.

[11] E. Ilavarasan, P. Thambidurai, and R. Mahilmannan. 2005. High performance task scheduling algorithm for heterogeneous computing system. In *Proceedings of the International Conference on Algorithms and Architectures for Parallel Processing*. Springer, 193–203.

[12] Nagarajan Kandasamy, John P. Hayes, and Brian T. Murray. 2005. Dependable communication synthesis for distributed embedded systems. *Reliabil. Eng. Syst. Safety* 89, 1 (2005), 81–92.

[13] Svilen Kanev, Kim Hazelwood, Gu-Yeon Wei, and David Brooks. 2014. Tradeoffs between power management and tail latency in warehouse-scale applications. In *Proceedings of the IEEE International Symposium on Workload Characterization (IISWC'14)*. IEEE, 31–40.

[14] Young Choon Lee and Albert Y. Zomaya. 2010. Energy conscious scheduling for distributed computing systems under different operating conditions. *IEEE Trans. Parallel Distrib. Syst.* 22, 8 (2010), 1374–1381.

[15] Srijeeta Maity, Anirban Ghose, Soumyajit Dey, and Swarnendu Biswas. 2021. Thermal-aware adaptive platform management for heterogeneous embedded systems. *ACM Trans. Embed. Comput. Syst.* 20, 5s (2021), 1–28.

[16] Sanjay Moulik, Rishabh Chaudhary, and Zinea Das. 2020. HEARS: A heterogeneous energy-aware real-time scheduler. *Microprocess. Microsyst.* 72 (2020), 102939.
[17] Sanjay Moulik, Arnab Sarkar, and Hemangee K. Kapoor. 2021. TARTS: A temperature-aware real-time deadline-partitioned fair scheduler. *J. Syst. Architect.* 112 (2021), 101847.
[18] Javier Pérez Rodríguez and Patrick Meumeu Yomsi. 2021. An efficient proactive thermal-aware scheduler for DVFS-enabled single-core processors. In *Proceedings of the 29th International Conference on Real-Time N/W and Systems*. 144–154.
[19] Sanjit Kumar Roy, Rajesh Devaraj, Arnab Sarkar, and Debabrata Senapati. 2021. SLAQA: Quality-level aware scheduling of task graphs on heterogeneous distributed systems. *ACM Trans. Embed. Comput. Syst.* 20, 5 (2021), 1–31.
[20] Shivashis Saha, Ying Lu, and Jitender S. Deogun. 2012. Thermal-constrained energy-aware partitioning for heterogeneous multi-core multiprocessor real-time systems. In *Proceedings of the IEEE International Conference on Embedded and Real-Time Computing Systems and Applications*. IEEE, 41–50.
[21] Debabrata Senapati, Arnab Sarkar, and Chandan Karfa. 2021. HMDS: A makespan minimizing DAG scheduler for heterogeneous distributed systems. *ACM Trans. Embed. Comput. Syst.* 20, 5s (2021), 1–26.
[22] Debabrata Senapati, Arnab Sarkar, and Chandan Karfa. 2022. PRESTO: A penalty-aware real-time scheduler for task graphs on heterogeneous platforms. *IEEE Trans. Comput.* 71, 2 (2022), 421–435. https://doi.org/10.1109/TC.2021.3052389
[23] Debabrata Senapati, Arnab Sarkar, and Chandan Karfa. 2023. Energy-aware real-time scheduling of multiple periodic DAGs on heterogeneous systems. *IEEE Trans. Comput.-Aided Design Integr. Circ. Syst.* 42, 8 (2023), 2447–2460. https://doi.org/10.1109/TCAD.2022.3228504
[24] Yanshul Sharma, Shounak Chakraborty, and Sanjay Moulik. 2022. ETA-HP: An energy and temperature-aware real-time scheduler for heterogeneous platforms. *J. Supercomput.* 78 (2022), 1–25.
[25] Hafiz Fahad Sheikh and Ishfaq Ahmad. 2011. Fast algorithms for thermal constrained performance optimization in DAG scheduling on multi-core processors. In *Proceedings of the International Green Computing Conference and Workshops*. IEEE, 1–8.
[26] Hafiz Fahad Sheikh and Ishfaq Ahmad. 2012. Fast algorithms for simultaneous optimization of performance, energy and temperature in DAG scheduling on multi-core processors. In *Proceedings of the International Conference on Parallel and Distributed Processing Techniques and Applications (PDPTA'12)*. 1.
[27] Hafiz Fahad Sheikh, Ishfaq Ahmad, Zhe Wang, and Sanjay Ranka. 2012. An overview and classification of thermal-aware scheduling techniques for multi-core processing systems. *Sustain. Comput.: Inform. Syst.* 2, 3 (2012), 151–169.
[28] Kevin Skadron. 2004. Hybrid architectural dynamic thermal management. In *Proceedings of the Design, Automation, and Test in Europe Conference and Exhibition*, Vol. 1. IEEE, 10–15.
[29] Kevin Skadron, Mircea R. Stan, Wei Huang, Sivakumar Velusamy, Karthik Sankaranarayanan, and David Tarjan. 2003. Temperature-aware microarchitecture. *ACM SIGARCH Comput. Architect. News* 31, 2 (2003), 2–13.
[30] H. Topcuoglu, S. Hariri, and Min-You Wu. 2002. Performance-effective and low-complexity task scheduling for heterogeneous computing. *IEEE Trans. Parallel Distrib. Syst.* 13, 3 (2002), 260–274.
[31] Jeffrey D. Ullman. 1975. NP-complete scheduling problems. *J. Comput. Syst. Sci.* 10, 3 (1975), 384–393.
[32] Guoqi Xie, Xiongren Xiao, Hao Peng, Renfa Li, and Keqin Li. 2022. A survey of low-energy parallel scheduling algorithms. *IEEE Trans. Sustain. Comput.* 7, 1 (2022), 27–46. https://doi.org/10.1109/TSUSC.2021.3057983
[33] Guoqi Xie, Gang Zeng, Renfa Li, and Keqin Li. 2017. Energy-aware processor merging algorithms for deadline constrained parallel applications in heterogeneous cloud computing. *IEEE Trans. Sustain. Comput.* 2, 2 (2017), 62–75.
[34] Lian-Tua Yeh, R. C. Chu, and W. S. Janna. 2003. Thermal management of microelectronic equipment: Heat transfer theory, analysis methods, and design practices. ASME press book series on electronic packaging. *Appl. Mech. Rev.* 56, 3 (2003), B46–B48.
[35] Xin Zhan, Reza Azimi, Svilen Kanev, David Brooks, and Sherief Reda. 2016. Carb: A c-state power management arbiter for latency-critical workloads. *IEEE Comput. Architect. Lett.* 16, 1 (2016), 6–9.
[36] Qingchen Zhang, Man Lin, Laurence T. Yang, Zhikui Chen, and Peng Li. 2019. Energy-efficient scheduling for real-time systems based on deep Q-learning model. *IEEE Trans. Sustain. Comput.* 4, 1 (2019), 132–141. https://doi.org/10.1109/TSUSC.2017.2743704
[37] Yi Zhao, Suzhi Cao, and Lei Yan. 2019. List scheduling algorithm based on pre-scheduling for heterogeneous computing. In *Proceedings of the IEEE International Conference on Parallel and Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing and Communications, Social Computing, and Networking (ISPA/BDCloud/SocialCom/SustainCom'19)*. IEEE, 588–595.
[38] Junlong Zhou, Kun Cao, Peijin Cong, Tongquan Wei, Mingsong Chen, Gongxuan Zhang, Jianming Yan, and Yue Ma. 2017. Reliability and temperature constrained task scheduling for makespan minimization on heterogeneous multi-core platforms. *J. Syst. Softw.* 133 (2017), 1–16.

Received 16 December 2022; revised 6 May 2023; accepted 12 August 2023